SymPlan: Enhancing Neurosymbolic Mathematical Reasoning in SLMs via Representation and Planning

Tay Bui[1,2], Huy N. G. Pham[1,2], Nhan Ha Le Thanh[3]
[1] University of Information Technology, Ho Chi Minh City, Vietnam
[2] Vietnam National University, Ho Chi Minh City, Vietnam
[3] FPT University, Ho Chi Minh City, Vietnam
{24521589, 24520690}@gm.uit.edu.vn, hnhan5355@gmail.com

***Abstract*—Mathematical reasoning with language models increasingly relies on executable symbolic programs, yet reliability remains elusive when compact models must translate a natural-language problem directly into code. The core difficulty is not only computation, but formulation: a syntactically valid program may still miss hidden constraints, choose an invalid branch, or collapse under the mixed grammar of prose, mathematics, and code. This limitation is particularly severe for small language models, causing monolithic approaches such as SymCode to exhibit frequent formulation, constraint-omission, and code-generation errors. We introduce SymPlan, a structured neurosymbolic framework that separates symbolic problem solving into structured problem representation, algorithmic planning, and plan-guided SymPy generation. By making variables, targets, and domain constraints explicit before code synthesis and carrying them into the generated program, SymPlan turns fragile one-shot generation into a more inspectable reasoning pipeline. Across MATH-500 and GSM8K, SymPlan consistently improves answer accuracy while maintaining high execution reliability, with particularly clear gains against monolithic SymPy generation. These results suggest that the next step for small language models is not merely larger computation, but more disciplined interfaces between informal mathematical intent and formal symbolic execution.**

***Index Terms*—Neurosymbolic Reasoning, Mathematical Problem Solving, Small Language Models, SymPy Code Generation, Structured Problem Representation, Algorithmic Planning.**

I. Introduction

Large Language Models (LLMs) have demonstrated strong capabilities in complex reasoning tasks, with Chain-of-Thought (CoT) prompting improving their ability to perform multi-step reasoning [1]. Subsequent methods improve robustness by sampling multiple reasoning paths, decomposing difficult questions into simpler subproblems, or constructing an explicit plan before solving [9]–[11]. However, autoregressive mathematical reasoning remains susceptible to calculation errors, missing reasoning steps, and semantic misunderstandings, making intermediate solutions difficult to reliably inspect or verify [7], [12].

Program-aided approaches address these limitations by delegating computation to executable programs. Program-Aided Language Models (PAL) [2] and Program-of-Thought (PoT) [3] separate reasoning from computation by using external interpreters for numerical operations. MathCoder and ToRA further integrate natural-language reasoning with code execution and mathematical tools [13], [14]. More recently, SymCode [4] extends this paradigm to symbolic mathematics by generating executable programs with the SymPy symbolic computation library [5], enabling mathematical solutions to be checked through program execution.

Despite these advances, program execution alone does not guarantee correct problem interpretation. Translation-based neurosymbolic systems likewise depend on the correctness of the generated formalization before a deterministic solver is invoked [12], [15]. Existing program-aided approaches often couple problem understanding, solution reasoning, and program synthesis within a single generation process. Consequently, a program may be syntactically valid and execute successfully while encoding an incorrect mathematical formulation. This issue is particularly challenging for compact language models, which must simultaneously interpret the problem, construct a solution strategy, and synthesize formal symbolic code. Although decomposition and planning can reduce missing-step errors [9], [11], [16], they have not been systematically connected to structured mathematical representation and SymPy generation for compact models.

To investigate these limitations, we conduct a systematic failure analysis across three compact language models—Qwen2.5-Coder-7B-Instruct [6], Qwen3-8B [17], and Llama-3.1-8B [18]—under 4-bit quantization on the full MATH-500 [7] and GSM8K [8] benchmarks, using SymPy [5] for program execution and evaluation. Across these models and benchmarks, we identify four recurring failure modes: syntax leakage, where formatting artifacts corrupt generated code; semantic misformulation, where executable programs encode an incorrect interpretation; symbolic API and type errors, involving incorrect handling of SymPy or Python objects; and constraint violations, where omitted domain conditions lead to invalid or extraneous solutions. These findings suggest that reliable symbolic reasoning requires addressing both program-level and problem-level errors rather than failures specific to a single model family or benchmark.

Motivated by these observations, we propose SymPlan, a

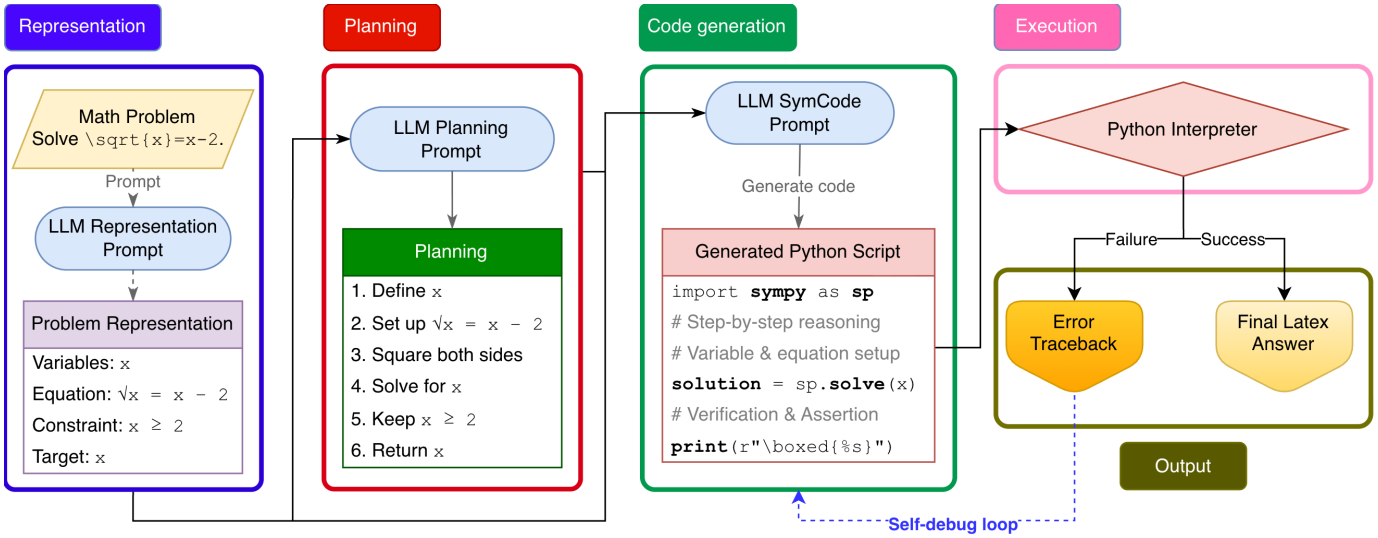

Fig. 1. Overview of SymPlan. A problem is transformed into a structured representation and an explicit plan before SymPy code generation. The script is executed by a Python interpreter; failures return a traceback to code generation, while successful runs produce a final LaTeX answer.

planning-guided neurosymbolic framework for compact language models. SymPlan separates symbolic problem solving into three stages: structured problem representation, algorithmic planning, and plan-guided SymPy generation. The structured representation captures variables, constants, targets, relationships, and domain constraints, allowing the model to determine what the problem represents before deciding how to solve it.

The generated script is then executed by a Python interpreter. Successful runs produce the final LaTeX answer, while execution failures return an error traceback to the code-generation stage.

Our contributions are twofold. First, we identify four recurring failure modes across three compact models on MATH-500 and GSM8K. Second, we introduce SymPlan, which separates structured problem representation from algorithmic planning before plan-guided SymPy generation, improving accuracy over Direct, CoT, and SymCode while maintaining high execution reliability.

II. Related Work

A. Mathematical Reasoning

Chain-of-Thought (CoT) prompting elicits intermediate natural-language rationales and substantially improves arithmetic, commonsense, and symbolic reasoning in sufficiently capable language models [1]. Zero-shot and few-shot extensions seek to make this reasoning more reliable without modifying model parameters. Self-consistency samples multiple reasoning trajectories and selects their majority answer [10], while Least-to-Most prompting first decomposes a difficult question into simpler dependent subproblems [11]. Plan-and-Solve similarly asks the model to construct and then execute a solution plan, reducing missing-step and calculation errors [9].

These methods improve answer accuracy, but their intermediate text is neither executable nor necessarily faithful to the computation that produced the final prediction [12]. This distinction is important in mathematics: a fluent derivation can contain an early semantic error, an omitted domain condition, or an invalid transformation whose consequences propagate through every later step. Process supervision and step-level verification can identify some local mistakes [7], but they require additional supervision or a capable verifier. SymPlan instead focuses on the interface between problem interpretation and symbolic execution by explicitly representing the mathematical state before generating either a plan or code.

B. Program-Aided Reasoning

Program-aided methods treat executable code as an intermediate representation, allowing an external interpreter to perform exact or repeated computation. PAL delegates computational steps to generated programs while retaining the model for semantic reasoning [2]; PoT similarly separates the formulation of a solution from numerical execution [3]. MathCoder interleaves natural language, code, and execution results through code-integrated mathematical training [13], whereas ToRA learns interactive trajectories that combine textual reasoning with calculators, computation libraries, and symbolic solvers [14]. These approaches demonstrate that tool use can reduce arithmetic burden and extend the range of operations available to a language model.

For symbolic mathematics, SymCode directly generates programs using SymPy [4], [5]. This supports exact algebra, calculus, equation solving, and modular arithmetic beyond ordinary numerical Python. However, executable code provides a certificate only for what the program actually computes. If the model encodes the wrong equation, returns an intermediate quantity, or omits a domain restriction, the interpreter may report success while the mathematical answer remains incorrect. Such risks grow for compact models because code generation

adds syntax, library selection, typing, and output-formatting requirements to the original reasoning task.

C. Neurosymbolic Formalization

Neurosymbolic frameworks use language models to translate informal inputs into formal structures that deterministic systems can execute. Faithful CoT separates translation into a symbolic reasoning chain from deterministic problem solving [12]. Logic-LM translates natural-language logic problems into solver-compatible formulations and uses solver feedback to refine invalid translations [15]. These systems improve faithfulness by making the solver's contribution explicit, but they also reveal a central bottleneck: deterministic inference cannot recover information that was omitted or mistranslated during formalization.

SymPlan shares this translation–execution perspective but targets heterogeneous mathematical problems and compact open-weight models. Rather than moving directly from the question to a complete formal program, it introduces two intermediate interfaces: a structured state containing variables, equations, constraints, and the target, followed by an implementation-independent algorithmic plan. This design aims to reduce semantic load before the model encounters the additional grammar and API requirements of SymPy code.

D. Planning and Verification

Task decomposition provides a complementary route to reliable reasoning. Decomposed Prompting delegates complex problems to modular subtask prompts [16]; Least-to-Most solves a sequence of increasingly dependent subproblems [11]; and Plan-and-Solve separates plan construction from plan execution [9]. Their plans are generally expressed in free-form language, however, and therefore do not by themselves guarantee that variables, constraints, and output targets survive the transition to executable code.

Execution feedback can repair a different class of failures. Self-Debugging shows that language models can inspect generated programs, execution results, and error messages to revise faulty code [19]. Logic-LM similarly uses symbolic-solver errors for self-refinement [15]. Such feedback is highly informative for syntax, API, and type errors, but a runnable program may provide no traceback even when its mathematical formulation is wrong. SymPlan therefore combines execution-guided repair with a fixed representation and plan: tracebacks may revise implementation details, while the explicit mathematical state anchors the intended semantics across debugging attempts.

III. Method

A. Framework Overview

Given a natural-language mathematical problem q, monolithic symbolic approaches such as SymCode [4] directly generate and execute a program, whereas other tool-integrated methods may interleave reasoning and program calls within a single trajectory [13], [14]:

$$C_{\text{base}} = \mathcal{M}(q), \quad \hat{y} = \text{Exec}(C_{\text{base}}) \tag{1}$$

This formulation requires the model to interpret the problem, select a solution strategy, and produce valid symbolic code simultaneously.

SymPlan instead separates these responsibilities into three model-guided stages, following the broader principle that explicit decomposition can reduce the burden of complex generation [11], [16]. It first constructs a formal representation $\mathcal{S}$ containing the quantities, target, equations, and domain constraints. It then derives an ordered solution plan $\mathcal{P}$ and uses both intermediate outputs to generate a SymPy program. The complete pipeline is defined as

$$\mathcal{S} = \text{Represent}_{\mathcal{M}}(q), \tag{2}$$

$$\mathcal{P} = \text{Plan}_{\mathcal{M}}(q, \mathcal{S}), \tag{3}$$

$$C_{\text{raw}} = \text{Generate}_{\mathcal{M}}(q, \mathcal{S}, \mathcal{P}), \tag{4}$$

$$o = \text{Exec}_{\text{Python}}(C_{\text{raw}}), \tag{5}$$

where o is either a successful result $\hat{y}$ or an execution traceback e. On success, $\hat{y}$ is formatted as the final LaTeX answer. On failure, e is returned to the code-generation prompt to produce a revised script. The representation, plan, and initial program are generated sequentially, making the problem formulation and solution strategy explicit before code synthesis.

B. Structured Problem Representation

Given the original problem q, a dedicated representation prompt asks the model to describe the mathematical state before proposing a solution. Its output is

$$\mathcal{S} = (\mathcal{V}, \mathcal{E}, \mathcal{C}, t), \tag{6}$$

where the fields correspond to:

- **Variables ($\mathcal{V}$):** Known and unknown quantities together with their mathematical roles.
- **Equations ($\mathcal{E}$):** Symbolic relations translated from the natural-language statement.
- **Constraints ($\mathcal{C}$):** Explicit and implicit conditions, including domains, integrality, positivity, bounds, and nonzero denominators.
- **Target (t):** The quantity or expression that the program must return.

This stage performs problem formulation rather than solution generation, reflecting the translation-first separation used in faithful neurosymbolic reasoning [12], [15]. In particular, recording constraints before algebraic transformations helps prevent valid-looking programs from accepting extraneous or out-of-domain solutions.

C. Algorithmic Planning

The planning prompt receives both q and $\mathcal{S}$ and produces an ordered sequence $\mathcal{P} = (p_1, \ldots, p_m)$. Each step specifies a mathematical operation, such as defining symbols, constructing equations, applying an algebraic transformation, solving for the target, filtering candidates using $\mathcal{C}$, and returning the requested result. The plan remains implementation-independent: it determines what must be computed and in which order, without yet producing Python syntax. This separation is related

to Plan-and-Solve and modular decomposition [9], [16], but conditions the plan on an explicit mathematical state. Consequently, code generation is conditioned on a solution strategy rather than deriving the strategy while writing the program.

D. Plan-Guided SymPy Generation

The code-generation prompt receives q, $\mathcal{S}$, and $\mathcal{P}$ and translates the planned operations into a self-contained Python script using SymPy [5]. Unlike approaches that generate code directly or interleave it with free-form reasoning [4], [13], [14], the script is constrained by both intermediate artifacts. It defines symbolic variables, encodes the governing equations, follows the plan in order, validates candidate solutions against the extracted constraints, and prints one final answer in LaTeX form. For the example $\sqrt{x} = x - 2$, the generated structure is:

```
import sympy as sp

x = sp.symbols('x', real=True)
eq = sp.Eq(sp.sqrt(x), x - 2)
candidates = sp.solve((x - 2)**2 - x, x)

valid = [s for s in candidates
         if s >= 2 and sp.simplify(eq.lhs.subs(x, s)
                                   - eq.rhs.subs(x, s))
     == 0]
print(r"\boxed{%s}" % sp.latex(valid[0]))
```

Here, the condition $x \geq 2$ comes from $\mathcal{C}$, while substitution into the original equation rejects roots introduced by squaring.

E. Execution and Debugging

The generated script $C^{(0)}$ is executed by a Python interpreter with a $\tau = 15$-second timeout. If execution succeeds and produces a parseable result, that result is returned as the final LaTeX answer. If execution raises an exception, the traceback $e^{(i)}$ and failed script $C^{(i)}$ are returned to the code-generation prompt:

$$C^{(i+1)} = \mathrm{Debug}_{\mathcal{M}}\big(q, \mathcal{S}, \mathcal{P}, C^{(i)}, e^{(i)}\big). \tag{7}$$

The representation and plan remain fixed during this loop so that debugging targets the implementation error without changing the intended mathematical formulation. This use of execution feedback follows prior self-debugging and solver-refinement work [15], [19], while restricting revision to the code-generation stage. Execution terminates when a valid output is obtained or the configured attempt limit is reached; unsuccessful runs are counted as execution failures.

IV. Experiments

A. Experimental Setup

1) Benchmarks: We evaluate all methods on two standard benchmarks:

- **MATH-500** [7]: A standardized 500-problem subset of the challenging MATH benchmark, categorized across 7 subjects (*Algebra, Intermediate Algebra, Number Theory, Counting & Probability, Geometry, Precalculus, Prealgebra*) and difficulty Levels 1 through 5.
- **GSM8K** [8]: The standard test split contains 1,319 grade-school mathematical word problems requiring multi-step arithmetic reasoning. For stratified analysis, we additionally group the test problems into three difficulty levels and six problem types: *Algebra, Arithmetic, Geometry, Measurement & Time, Percentages & Finance*, and *Ratios & Rates*.

2) Model and Execution Configuration: We evaluate three compact backbones: **Qwen2.5-Coder-7B-Instruct** [6], **Qwen3-8B** [17], and **Llama-3.1-8B-Instruct** [18]. This selection includes a code-specialized model, a stronger general-purpose reasoning model, and a widely used open-weight baseline. All models are quantized to 4-bit NormalFloat4 (NF4) via `bitsandbytes` with Scaled Dot-Product Attention (SDPA). Generation is deterministic across all methods: greedy search with `temperature` $= 0.0$, `top_p` $= 1.0$, and `max_new_tokens` $= 1024$. The execution timeout is set to $\tau = 15$ seconds.

3) Baseline Methods: We benchmark SymPlan against three baselines under identical runtime environments:

1) **Direct:** Zero-shot direct answer generation without intermediate steps.
2) **Chain-of-Thought (CoT):** Zero-shot natural-language step-by-step reasoning [1].
3) **SymCode:** Monolithic CoT and SymPy script synthesis [4].

4) Evaluation Metrics:

- **Accuracy (% Exact Match):** Evaluated through three-tier equivalence matching: string canonicalization, SymPy algebraic simplification $\mathrm{simplify}(\hat{y} - y) = 0$, and float tolerance $|\hat{y} - y| < 10^{-5}$.
- **Execution Success Rate (ESR %):** The percentage of generated programs that complete within the timeout without runtime exceptions and emit a parseable boxed result (applicable only to program-aided methods).

B. Main Results

Table I summarizes cross-model answer accuracy on MATH-500 and GSM8K, while Fig. 2 reports execution success separately for the program-aided methods.

The cross-model results expose a consistent limitation of monolithic program generation in small models. CoT exceeds SymCode on both benchmarks for all three backbones.

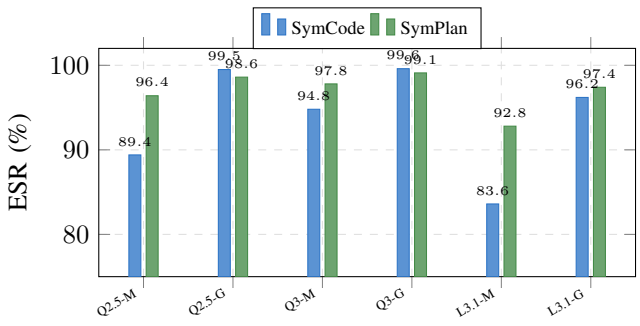

Fig. 2. Execution Success Rate (ESR) across models and benchmarks. M and G denote MATH-500 and GSM8K, respectively.

TABLE I
CROSS-MODEL ANSWER ACCURACY (%) ON MATH-500 AND GSM8K.

Model	Method	MATH-500	GSM8K
Qwen2.5-7B	Direct	21.80	25.23
	CoT [1]	65.80	87.11
	SymCode [4]	59.72	85.83
	SymPlan	**74.20**	**92.40**
Qwen3-8B	Direct	27.40	35.60
	CoT [1]	71.80	90.60
	SymCode [4]	68.20	89.90
	SymPlan	**79.60**	**94.80**
Llama-3.1-8B	Direct	18.60	22.10
	CoT [1]	58.40	80.30
	SymCode [4]	50.80	77.90
	SymPlan	**66.90**	**86.70**

TABLE II
ACCURACY (%) BY DIFFICULTY LEVEL USING QWEN2.5-CODER-7B-INSTRUCT.

Dataset	Level	Direct	CoT	SymCode	SymPlan
MATH-500	1	44.19	88.37	93.02	**95.35**
	2	35.56	83.33	78.89	**90.00**
	3	27.62	78.10	69.52	**83.81**
	4	10.94	60.94	43.33	**70.31**
	5	11.19	41.79	35.82	**52.99**
GSM8K	1	45.17	94.08	90.65	**96.26**
	2	21.12	87.99	86.54	**93.17**
	3	10.00	77.93	79.31	**86.90**

The gaps are 6.08 and 1.28 percentage points for Qwen2.5-Coder-7B, 3.60 and 0.70 points for Qwen3-8B, and 7.60 and 2.40 points for Llama-3.1-8B on MATH-500 and GSM8K, respectively. Nevertheless, SymCode frequently obtains ESR values above 90%, demonstrating that a runnable program can still encode an incorrect equation, omit a condition, or return the wrong quantity. This observation is consistent with the SymCode study, which reports that its gains depend strongly on the coding proficiency of the base model and that less code-optimized models may underperform prose-based reasoning [4].

SymPlan achieves the highest accuracy for every evaluated backbone. Relative to CoT, its gains are 8.40 and 5.29 points for Qwen2.5-Coder-7B, 7.80 and 4.20 points for Qwen3-8B, and 8.50 and 6.40 points for Llama-3.1-8B. Qwen3-8B provides the strongest absolute results, reaching 79.60% on MATH-500 and 94.80% on GSM8K, while Llama-3.1-8B produces the lowest scores across methods. Importantly, the SymPlan improvement persists across all three models, suggesting that representation and planning reduce the formulation burden before code synthesis rather than merely exploiting the coding strength of one backbone.

Table II shows that SymPlan remains strongest across the reported difficulty levels. Among the baselines, however, the relative ordering is not uniform: SymCode leads CoT on MATH Level 1 and GSM8K Level 3, whereas CoT is stronger on the remaining levels. Thus, symbolic code is beneficial when the model forms the correct program, while free-form CoT can be more robust when code synthesis introduces additional failure opportunities. SymPlan reduces this sensitivity by fixing the representation and solution plan before code generation.

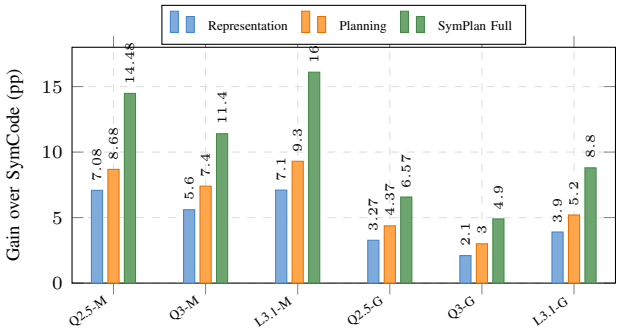

Fig. 3. Accuracy gains over SymCode [4] in the component ablation. M and G denote MATH-500 and GSM8K, respectively; single-component variants retain only representation or planning.

The subject-wise results in Table III show that the improvements extend beyond the aggregate scores. On MATH-500, the largest gains over CoT occur in Intermediate Algebra and Geometry, where explicit equations and domain conditions are especially important. On GSM8K, the clearest improvements appear in Percentages & Finance and Arithmetic. Geometry reaches the same ceiling as CoT, while SymPlan remains strongest or tied across all reported subjects.

C. Ablation Study

We ablate structured representation and algorithmic planning independently on both benchmarks and all three backbones to measure their individual and combined effects.

Fig. 3 shows that both components improve over SymCode [4] for every model–dataset pair, with planning providing the stronger individual variant in all six settings. The full pipeline achieves the highest accuracy throughout. Relative to SymCode, its gains are 14.48 and 6.57 points for Qwen2.5-7B, 11.40 and 4.90 points for Qwen3-8B, and 16.10 and 8.80 points for Llama3.1-8B on MATH-500 and GSM8K, respectively. The larger improvements on MATH-500 suggest that explicit representation and planning are especially useful when problems require longer symbolic transformations and stricter constraint handling. Their consistent combination gains indicate complementary roles: representation stabilizes the mathematical state, whereas planning organizes that state into an executable solution strategy.

D. Error Analysis and Qualitative Examples

The paired Level-5 logs provide direct evidence for the central hypothesis of this work. On the same 134 MATH-500 problems, CoT answers 56 correctly (41.79%), whereas

TABLE III
SUBJECT-WISE ACCURACY (%) USING QWEN2.5-7B. THE SYMPLAN COLUMN IS HIGHLIGHTED.

MATH-500

Subject	Direct	CoT	SymCode	SymPlan
Algebra	19.35	83.06	68.52	**87.90**
Counting & Probability	21.05	68.42	66.67	**78.95**
Geometry	12.20	46.34	40.00	**58.54**
Intermediate Algebra	18.56	47.42	46.99	**59.79**
Number Theory	32.26	80.65	80.39	**87.10**
Prealgebra	28.05	74.39	70.42	**82.93**
Precalculus	19.64	42.86	35.29	**50.00**

GSM8K

Subject	Direct	CoT	SymCode	SymPlan
Algebra	22.00	87.33	87.33	**89.33**
Arithmetic	27.39	87.10	86.22	**92.93**
Geometry	6.67	**100.00**	80.00	**100.00**
Measurement & Time	28.00	93.00	88.00	**97.00**
Percentages & Finance	21.94	81.63	81.63	**89.80**
Ratios & Rates	23.88	91.04	89.55	**94.03**

Problem: How many distinct values can be obtained by inserting parentheses into $2 \cdot 3 \cdot 4 \cdot 5 + 1$, without rearranging terms?

Observed SymCode Execution:
- Attempt 1 calls `evalf()` on a Python integer and raises an API error.
- The traceback-guided retry evaluates only the original expression: $2 \cdot 3 \cdot 4 \cdot 5 + 1 = 121$.
- The script executes and passes verification, but never enumerates alternative parenthesizations.
- **Verdict:** INCORRECT (output: 121; gold: 4).

SymPlan Reconstruction:
- *Representation:* Preserve operand order, fixed operators, and the target "number of distinct values."
- *Planning:* Enumerate all valid binary parenthesizations and collect unique results.
- *Execution:* Obtain $\{121, 122, 126, 144\}$ and return its cardinality.
- **Verdict:** CORRECT (illustrative output: 4).

Fig. 4. A successful execution does not imply a correct mathematical formulation. The SymCode behavior is taken from the Level-5 log; the SymPlan path is a qualitative reconstruction.

SymCode answers 48 correctly (35.82%). SymCode nevertheless executes successfully on 110 problems, corresponding to an ESR of 82.09%. Among these successful executions, 62 (56.36%) still produce an incorrect answer, and 21 incorrect outputs are even accepted by the lightweight verifier. Thus, execution success primarily measures program validity, not whether the generated program represents the intended mathematics.

Repairing execution without repairing meaning. Fig. 4 shows that traceback feedback repairs the API failure but leaves the semantic error untouched. The retry produces executable code by simplifying the task into direct evaluation, whereas representation and planning preserve the combinatorial target before code synthesis.

Preserving structural conditions. For the degree-5 interpolation problem with $p(n) = n/(n^2 - 1)$ for $n = 2, \ldots, 7$, SymCode produces an executable but unrelated polynomial construction and returns 39690 instead of $3/56$. The error is not numerical; the code fails to preserve the six interpolation constraints and the degree condition. A structured representation exposes these six exact point constraints, after which the plan can apply Lagrange interpolation or symbolic interpolation with rational arithmetic before evaluating $p(8)$.

Maintaining variable semantics and domains. For the number of real x such that $\sqrt{120 - \sqrt{x}}$ is an integer, SymCode runs successfully, returns 0, and passes verification because the verifier only checks that the output is a nonnegative integer. SymPlan instead introduces $n = \sqrt{120 - \sqrt{x}} \in \mathbb{Z}_{\geq 0}$, derives $\sqrt{x} = 120 - n^2 \geq 0$, and obtains $n \in \{0, \ldots, 10\}$. This yields 11 valid values of x, while making the domain conditions explicit before code generation.

These cases explain why CoT can outperform monolithic SymCode on compact models despite SymCode's high ESR. Traceback-guided repair is effective for syntax, API, and type failures, but it cannot reliably detect a runnable program that encodes the wrong target, transformation, or constraint. The reconstructed SymPlan examples illustrate the intended mechanism: representation stabilizes the mathematical state, planning selects a valid transformation sequence, and symbolic execution is used only after these decisions have been made. These illustrative SymPlan outputs should be replaced or confirmed when the corresponding runs finish.

V. CONCLUSION

In this paper, we presented SymPlan, a structured neurosymbolic framework for mathematical reasoning in compact open-weight language models. By separating formal problem representation from algorithmic planning before plan-guided SymPy generation, SymPlan addresses semantic misformulation and constraint omission through an explicit reasoning pipeline. The generated program carries the extracted constraints into Python execution, with interpreter tracebacks supporting correction when execution fails. Evaluations on MATH-500 and GSM8K demonstrate higher mathematical accuracy while maintaining high execution reliability, without parameter fine-tuning.

In future work, we plan to extend SymPlan to multimodal mathematical reasoning involving geometric diagrams, as well as investigate the integration of interactive formal theorem provers (such as Lean) to verify open-ended deductive proofs on edge devices.

REFERENCES

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[2] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, vol. 202, 2023, pp. 10 764–10 799.

[3] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Transactions on Machine Learning Research*, 2023.

[4] S. B. Nezhad, Y. Li, and A. Agrawal, "SymCode: A neurosymbolic approach to mathematical reasoning via verifiable code generation," in *Findings of the Association for Computational Linguistics: EACL 2026*, 2026, pp. 1489–1503.

[5] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh *et al.*, "SymPy: Symbolic computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017.

[6] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[7] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[9] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, vol. 1, 2023, pp. 2609–2634.

[10] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[11] D. Zhou, N. Sch"arli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, and E. H. Chi, "Least-to-most prompting enables complex reasoning in large language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, "Faithful chain-of-thought reasoning," in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2023.

[13] K. Wang, H. Ren, A. Zhou, Z. Lu, S. Luo, W. Shi, R. Zhang, L. Song, M. Zhan, and H. Li, "MathCoder: Seamless code integration in large language models for enhanced mathematical reasoning," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, "ToRA: A tool-integrated reasoning agent for mathematical problem solving," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023, pp. 3806–3824.

[16] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed prompting: A modular approach for solving complex tasks," in *International Conference on Learning Representations (ICLR)*, 2023.

[17] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[18] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[19] X. Chen, M. Lin, N. Sch"arli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.